

TonCrown — independent audit scope

Prepared for external auditors. Everything an auditor needs to quote and start.

What is deployed

Contract	EQBHG-1-XAsYnMGmX8ecP7fU_AM8oSV-r6ZQtYBD4sEGVdPy (TON mainnet)
Code hash	4fb7a27304061ad0cf13ce47677759355843a68f93ad07d413b8b62a3c4ae69d
Source	contracts/ton_crown.tact — 2095 lines, Tact
Commit	0b2111edfbe0ff7dd199638d8db56dffa12054dc6
Repository	https://github.com/Toncrown/contract, branch claude/ton-contract-refund-bug-3q464f
Live since	31 August 2026
Users / value	106 users, 222.4 TON of staked principal owed

The deployed code hash can be verified against a local build with

```
npx blueprint run identifyDeployed.
```

What the contract does

A referral and staking platform. Users buy sequential levels (1–10, 1.25–12.55 TON). Each purchase is split among four creator wallets, up to two referrers, and a per-level spillover pool. Levels 4+ unlock VIP tiers that allow TON and USDT staking at 0.95–1.95% daily ROI. Users accrue a 0.01 TON daily check-in reward, claimable on demand.

Referral structure: each user has at most 6 direct referrals. Beyond that, new signups are placed elsewhere in the sponsor's subtree, and the referral share splits between the

link referrer (whose link was used) and the *slot* referrer (the matrix parent).

Priority areas

Ranked by what would hurt most. The first is where a live bug was already found.

1. **Message value accounting.** The production incident that prompted this work was `self.reply()` compiling to `SendRemainingValue` (mode 64) in a receiver that had already spent the inbound value, refunding buyers their entire payment whenever the balance could cover it. Please re-derive this class independently across every receiver and send mode.
2. **Solvency and payout ordering.** Whether any path can commit state and then fail to pay, or pay twice. Particular attention to `processRewardPayout`, which pays a staking reward and returns principal in the same transaction.
3. **Referral and spillover economics.** Whether a user can direct value to themselves, inflate their own position, or starve the spillover rotation.

4. **Gas ceilings.** Several loops are bounded because unbounded versions hit the 1,000,000 limit. Please check the bounds hold as user count and per-user stake count grow, including in get methods.
5. **Owner powers.** `OwnerWithdraw`, `UpgradeContract` (SETCODE), `ChangeOwner`, `SetPaused`, `SetStakeCapitalPolicy`, and the import receivers. We want the blast radius documented, not necessarily reduced.
6. **Upgrade safety.** `UpgradeContract` swaps code but not storage. We would like guidance on what storage changes are safe to ship this way.
7. **Jetton (USDT) handling.** `JettonTransferNotification` parsing, the refund path for rejected deposits, and whether jettons can be stranded or double-credited.

Known issues — disclosed, not to be re-litigated as findings

Please confirm the fixes rather than rediscover the bugs. All were found and fixed in this codebase before deployment; regression tests exist for each.

1. `self.reply()` after spending inbound value refunded the buyer's whole payment.
2. Referrer was read from every incoming message, so a caller could redirect the referral share to any address, including their own.
3. A matured stake could be closed without paying out the principal, because the reward and capital affordability checks were separate and both sends used `SendIgnoreErrors`.
4. `myBalance()` includes the inbound message value, so affordability checks overstated the treasury.
5. The spillover scan was unbounded and hit exit `-14` at roughly 200 users.
6. A failed jetton notification stranded USDT with no record.
7. `isPaused` was read but could never be set.
8. `getTreasuryLiabilities` exceeded the get-method gas limit at a page size of 50.

Accepted design decisions

Not bugs; do not report as findings, but do tell us if the risk is worse than we think.

- `forwardStakeCapital = true`. Staking capital is forwarded to `creatorWallet3` on deposit while the contract retains the obligation to pay ROI and return principal. The contract is therefore structurally short by the staked balance and solvency depends on manual funding. `SetStakeCapitalPolicy{false}` would make refunds self-funding.
- **The owner is trusted.** It can withdraw the balance, replace the code and transfer ownership.
- **A rejected jetton deposit costs the contract ~0.05 TON to refund.** Griefing costs the attacker more than it drains.

Measured gas ceilings

Measured against real data; useful as a baseline to check against.

Path	Result
<code>UpgradeLevel</code>	117k gas peak at 1000 users, flat
<code>getTreasuryLiabilities</code>	page 25 max, 30 fails
<code>getAdminUserSnapshotsPaginated</code>	page 20 max
<code>getUserListPaginated</code>	fine at 50

What we provide

- Full source and git history.
- 6 test files covering regressions, migration, upgradeability, money flows and referral splits. Run with `npx jest`.
- `MIGRATION.md` and `MIGRATION-REPORT.md` for how the contract was deployed and populated.
- `contracts/legacy_v1.tact` — the previous deployed version, for comparison.
- A testnet deployment carrying a copy of real production data:
`EQBcOWdhOLCipauoIne1F2w9Fd1bR00gnXTLAh8_a3Alh9Jx`.

Deliverables requested

1. Findings by severity, each with a concrete exploit path.
2. Confirmation that the eight known issues above are genuinely fixed.
3. A written opinion on upgrade safety: which storage changes `UpgradeContract` can carry.
4. Re-review after fixes.

Notes for quoting

The contract is live and holds user funds, so please state your lead time. A fix can be shipped to the same address via `UpgradeContract` without migrating users, so findings are actionable quickly.

This codebase has had one review pass, by a single reviewer, which found eight issues in code that had been running in production for months. Treat it as unaudited.